

General Instructions

General Guidelines

Check that the "signature.txt" file is present at the root of the cloned repository.	\$ shasum Born2beRoot.vdi
As a precaution, you may duplicate the original virtual machine to keep a copy.	

Mandatory Part

Project Overview

The basic functioning of their virtual machine	A virtual machine is a "computer inside a computer". A system that uses the hardware from the real computer to create another virtual environment. You can create virtual machines using an hypervisor software such as VirtualBox.
Their choice of operating system	I chose Debian because it's a system easier to administrate for begginers, with a lot of documentation and a large community.
The basic differences between Rocky and Debian	Stability and Use Case: <u>Debian</u> : Highly stable, suitable for servers, desktops, general use; good for beginners and advanced users. <u>Rocky Linux</u> : Stable, enterprise-focused, ideal for environments needing RHEL compatibility. Package Management: <u>Debian</u> : APT with .deb packages; user-friendly. <u>Rocky Linux</u> : YUM/DNF with .rpm packages; aligned with enterprise standards. Community and Support: <u>Debian</u> : Large, well-established community with extensive resources. <u>Rocky Linux</u> : Rapidly growing community, replicating CentOS support structure. Documentation and Learning Curve: <u>Debian</u> : Extensive documentation, approachable for beginners. <u>Rocky Linux</u> : Good but less extensive documentation; may require more effort for new users.
The benefits of virtual machines	Cost-Effective: No need to buy extra hardware; you can use your existing computer. Flexibility: You can run different operating systems and software on the same computer. Security: VMs are isolated from each other, so problems in one don't affect the others. Testing: You can use VMs to test new software or updates without risking your main system. Convenience: Easily create, modify, and delete virtual machines as needed.
If the chosen operating system is Debian, they must explain the difference between aptitude and apt, and what APPArmor is. During the defense, a script must display information every 10 minutes. Its operation will be checked in detail later. If the explanations are not clear, the evaluation stops here.	aptitude and apt are both tools for managing packages in Debian. aptitude : Remembers which packages were explicitly requested and which were installed as dependencies. Automatically uninstalls packages that were only dependencies when they are no longer needed. Offers a more sophisticated interface, including a text-based GUI. apt : Performs only the actions specified in the command line. Simpler and faster for straightforward package management tasks. APPArmor : is a Linux security module that provides Mandatory Access Control (MAC). It restricts the capabilities of programs by defining what files they can read, write, and execute. This helps to limit the potential damage from compromised or malicious software. APPArmor is included by default in Debian. Check with the command: aa-status.

Simple Configuration

Verify that the machine does not have a graphical environment at startup. A password will be requested before any connection attempt on this machine. Log in with a non-root user. Verify that the chosen password respects the rules imposed in the subject. Then, with the help of the evaluator, make sure that:	\$ ls /usr/bin/*session → /usr/bin/dbus-run-session (no gnome-sessionls, xfce4-sessionls, lxsessionls, startkde, startxfce4, etc)
The chosen operating system is indeed Debian or Rocky	\$ hostnamectl
The UFW service is properly launched	\$ sudo service ufw status
The SSH service is properly launched. If something does not work as expected or is not clearly explained, the evaluation stops here.	\$ sudo service ssh status

User

The subject requires that a user with the login name of the evaluated person be present on the virtual machine. Verify that this user has been added and that they belong to the "sudo" and "user42" groups.	\$ getent group sudo \$ getent group user42
Ensure the rules imposed in the subject concerning the password policy have been implemented by following the steps below.	
First, create a new user. Assign them a password of your choice, respecting the subject's rules.	\$ sudo adduser <username>
The evaluated person must now explain how they were able to implement the requested rules on their virtual machine.	\$ sudo nano /etc/pam.d/common-password → check password rules \$ sudo nano /etc/login.defs → expiration rules
Normally, one or two files would have been modified.	
Now that you have a new user, ask the evaluated person to create a group named "evaluating" in front of you and assign it to this user.	\$ sudo groupadd evaluating \$ sudo usermod -aG <username> evaluating
Finally, check that this user indeed belongs to the "evaluating" group.	\$ getent group evaluating
To finish, ask the evaluated person to explain the purpose of this password policy, as well as the advantages and disadvantages of its implementation. Of course, answering that it is because the subject requires it does not count.	I installed the libpam-pwquality library and added the rules for the password. This policy is used to prevent attacks by enforcing strong and regularly updated passwords. It increases security; however, it might be inconvenient for users who constantly need to change and remember new passwords.

Hostname and Partitions

Verify that the hostname of the machine is formatted as follows: login42 (login of the evaluated person).	\$ hostnamectl
Change this hostname by replacing the login with yours, then restart the machine. If upon restarting, the hostname has not been updated, the evaluation stops here.	\$ sudo nano /etc/hostname \$ sudo nano /etc/hosts \$ sudo reboot
You can now restore the original hostname of the machine.	*repeat*
Ask the evaluated person how to display the partitions of their virtual machine.	\$ lsblk
Compare the output with the example given in the subject. Note: if the evaluated person has done the bonuses, you should refer to the example from the bonus part.	

This part is an opportunity to discuss partitions! The evaluated person must give you a brief explanation of how LVM works and why it's beneficial.	LVM (Logical Volume Manager) provides flexible disk space management, which is especially useful for servers where storage needs can change over time. Physical Volume: actual hard drives or storage devices. They can be entire drives or partitions and are added to a storage pool using LVM commands. Volume Group: a big storage pool created by combining multiple physical volumes. Logical Volume: partitions created from the volume group. The main advantage of LVM is that you can resize logical volumes anytime without losing data. If you need more space, you can add more physical volumes to the volume group. LVM also allows you to take "snapshots" of a logical volume, capturing its state at a specific time, which is useful for backups and testing. Encrypted Partition: part of your storage that is secured with encryption, making the data unreadable without the correct decryption key. Tools like LUKS are used for this purpose.
--	---

SUDO

Verify that the "sudo" program is indeed installed on the virtual machine.	\$ dpkg -s sudo
The evaluated person must now show how to assign your new user to the "sudo" group.	\$ sudo usermod -aG sudo <username>
The subject imposes strict rules for sudo. The evaluated person must first explain the purpose and operation of sudo with examples of their choice. Then, they must demonstrate the implementation of the rules imposed by the subject.	\$ sudo nano /etc/sudoers
Verify that the "/var/log/sudo/" directory exists and contains at least one file. Check the contents of the files in this directory; you should see a history of commands used with sudo. Finally, attempt to run a command via sudo. Check if the file(s) in the "/var/log/sudo/" directory have been updated. If something does not work as expected or is not clearly explained, the evaluation stops here.	\$ sudo cat /var/log/sudo/sudo.log

UFW / FirewallD

Verify that the "UFW" (or "Firewalld" for rocky) program is indeed installed on the virtual machine.	\$ dpkg -s ufw
The evaluated person must give you a basic explanation of what UFW (or "Firewalld") is and why it's important to use it.	\$ sudo service ufw status UFW (Uncomplicated Firewall) is a simple interface to manage the firewall of a device, making it easier to modify firewall rules without compromising security. You use it to configure which ports to allow connections to and which ports to block. This is particularly useful for services like SSH, as you can specify which port SSH should use and control access to it. Importance: <u>Security:</u> Helps protect your system by controlling incoming and outgoing network traffic. <u>Ease of Use:</u> Simplifies firewall management with an easy-to-understand syntax. <u>Flexibility:</u> Allows you to easily configure rules for specific services and ports.
List the active rules in UFW (or "Firewalld"). There should be a rule for port 4242.	\$ sudo ufw status numbered
Add a new rule to open port 8080. Verify that it has been added by listing the active rules.	\$ sudo ufw allow 8080
	\$ sudo ufw status numbered
Finally, with the help of the evaluated person, delete this new rule. If something does not work as expected or is not clearly explained, the evaluation stops here.	\$ sudo ufw delete <rule number>

SSH

Verify that the SSH service is indeed installed on the virtual machine.	dpkg -s ssh
Verify that it is functioning correctly.	sudo service ssh status
The evaluated person must give you a basic explanation of what SSH is and why it's important to use it.	SSH (Secure Shell) is a protocol used to securely connect to a remote computer over a network. It uses encryption to ensure that all communication between the client and the host is secure and protected from eavesdropping. Key Points: <u>Authentication:</u> SSH authenticates the identity of the client and the host to establish a secure connection. <u>Encryption:</u> All data transferred between the client and the host is encrypted, ensuring privacy and security. <u>Usage:</u> Users on Mac or Linux can use the terminal to connect to a server via SSH. Windows users can use tools like PuTTY or the built-in OpenSSH client. Importance: <u>Security:</u> Protects sensitive data and commands transmitted over potentially insecure networks. <u>Remote Access:</u> Allows users to remotely manage servers, perform administrative tasks, and securely transfer files. <u>Convenience:</u> Enables secure, remote access to systems, facilitating system administration and development tasks.
Verify that the SSH service uses only port 4242.	ssh carlos-j@<ip> -p <any other port>
The evaluated person must help you use SSH to connect with the new user created. For this, you can use a key or a simple password, depending on the evaluated person. Of course, you must ensure that SSH cannot be used with the "root" user as indicated in the subject. If something does not work as expected or is not clearly explained, the evaluation stops here.	ssh root@<ip> -p 4242 -> access denied
	ssh <username>@<ip> -p 4242

Script Monitoring

The operation of their script by displaying its code	<pre>#!/bin/bash # Architecture and Kernel version arc=\$(uname -s) # Number of physical processors pcpu=\$(grep "physical id" /proc/cpuinfo sort uniq wc -l) # Number of virtual processors vcpu=\$(grep "^processor" /proc/cpuinfo wc -l) # Available and used RAM fram=\$(free -m awk '\$1 == "Mem:" {print \$2}') uram=\$(free -m awk '\$1 == "Mem:" {print \$3}') aram=\$(free -m awk '\$1 == "Mem:" {print \$7}') pram=\$(free awk '\$1 == "Mem:" {printf("%.2f"), \$3/\$2*100}') # Disk space calculations in MB fdisk=\$(df -BM --total grep '^total' awk '{print \$2}' sed 's/M//') udisk=\$(df -BM --total grep '^total' awk '{print \$3}' sed 's/M//') adisk=\$(df -BM --total grep '^total' awk '{print \$4}' sed 's/M//') pdisk=\$(df -BM --total grep '^total' awk '{printf("%.2f"), \$3/\$2*100}') # Convert MB to GB for display with two decimal places using awk fdisk_gb=\$(awk "BEGIN {printf \"%.2f\", \$fdisk/1024}") udisk_gb=\$(awk "BEGIN {printf \"%.2f\", \$udisk/1024}") adisk_gb=\$(awk "BEGIN {printf \"%.2f\", \$adisk/1024}") # CPU load cpul=\$(top -bn1 grep '^%Cpu' cut -c 9- xargs awk '{printf("%.1f%%"), \$1 + \$3}') # Last reboot lb=\$(who -b awk '\$1 == "system" {print \$3 " " \$4 " " \$5}') lb_formatted=\$(date -d "\$lb" +%Y-%m-%d %H:%M) # LVM usage lvmu=\$(if [\$(lsblk grep "lvm" wc -l) -eq 0]; then echo no; else echo yes; fi) # Active TCP connections ctcp=\$(ss -neopt state established wc -l) # Number of users ulog=\$(users wc -w) # IPv4 and MAC address ip=\$(hostname -i awk '{print \$1}') mac=\$(ip link show grep "ether" awk '{print \$2}') # Number of sudo commands executed cmds=\$(journalctl _COMM=sudo grep COMMAND wc -l) # Output results wall " #Architecture: \$arc #CPU physical: \$pcpu #vCPU: \$vcpu #Memory Usage: \$uram/\$framMB (\$pram%) - Available: \${aram}MB #Disk Usage: \$(udisk_gb)GB/\$(fdisk_gb)GB (\$pdisk%) - Available: \${adisk_gb}GB #CPU load: \$cpul #Last boot: \$lb_formatted #LVM use: \$lvmu #Connections TCP: \$ctcp ESTABLISHED #User log: \$ulog #Network: IP: \$ip MAC: \$mac #Sudo: \$cmds cmd" monitoring_helper.sh #!/bin/bash # Function to calculate minutes since boot calculate_minutes_since_boot() { boot_times=\$(who -b awk '\$1 == "system" {print \$4 " " \$5}') boot_epoch=\$(date -d "\$boot_time" +%s) current_epoch=\$(date +%s) minutes_since_boot=\$(((current_epoch - boot_epoch) / 60)) echo "\$minutes_since_boot" } # Interval in minutes interval_minutes=10 # Calculate minutes since boot minutes_since_boot=\$(calculate_minutes_since_boot) # Check if the current time is a multiple of the interval since boot if [\$((minutes_since_boot % interval_minutes)) -eq 0]; then /usr/local/bin/monitoring.sh fi</pre>	This line specifies that the script should be executed using the Bash shell. uname -a: Fetches system information including the kernel version, machine hardware name, processor type, etc. grep "physical id" /proc/cpuinfo: Extracts lines containing "physical id" from the CPU information file. sort uniq: Sorts and removes duplicate lines. wc -l: Counts the number of unique physical processors. grep "^processor" /proc/cpuinfo: Extracts lines starting with "processor". wc -l: Counts the number of virtual processors (logical cores). free -m: Displays memory usage in megabytes. awk '\$1 == "Mem:" {print \$2}': Extracts total memory (fram), used memory (uram), and available memory (aram). awk '\$1 == "Mem:" {printf("%.2f"), \$3/\$2*100}': Calculates the percentage of used memory (pram). df -BM --total: Displays disk usage in megabytes, summarizing all filesystems. grep '^total': Extracts the total line. awk '{print \$2}' sed 's/M//': Extracts and cleans up total (fdisk), used (udisk), and available (adisk) disk space. awk '{printf("%.2f"), \$3/\$2*100}': Calculates the percentage of used disk space (pdisk). Converts the disk space from megabytes to gigabytes for better readability. top -bn1: Runs top command in batch mode for a single iteration. grep '^%Cpu': Extracts CPU usage line. cut -c 9- xargs awk '{printf("%.1f%%"), \$1 + \$3}': Processes the CPU usage to calculate the total load (user + system) and formats it as a percentage. who -b: Displays the last boot time. awk '\$1 == "system" {print \$3 " " \$4 " " \$5}': Extracts the boot time. date -d "\$lb" +%Y-%m-%d %H:%M: Formats the boot time. lsblk grep "lvm" wc -l: Checks for the presence of LVM volumes. Conditional to set lvmu to "yes" if LVM volumes are found, otherwise "no". ss -neopt state established: Lists established TCP connections. wc -l: Counts the number of established connections. users: Lists logged-in users. wc -w: Counts the number of logged-in users. hostname -i awk '{print \$1}': Retrieves the IPv4 address. ip link show grep "ether" awk '{print \$2}': Retrieves the MAC address. journalctl _COMM=sudo grep COMMAND: Filters the system logs for sudo commands. wc -l: Counts the number of sudo commands executed.
What "cron" is	Cron is a command line utility used to schedule commands or scripts to run automatically at specified intervals or at a specific time. It is particularly useful for automating repetitive tasks, such as running backups, sending emails, or restarting the server at a specific time each day.	
How they have set up their script to execute every 10 minutes from server launch. Once the proper functioning of the script is verified, the evaluated person must make sure that this script executes every minute. You can launch whatever you wish to ensure that the script runs correctly with dynamic values. Finally, the evaluated person must ensure that the script no longer runs at server launch, without modifying the script itself. To verify this point, the server will need to be restarted one last time. At startup, it must be verified that the script still exists in the same place, that its permissions have remained unchanged, and that it has not been modified. If any of the above points are not correct, the evaluation stops here.	<pre>sudo crontab -u root -e sudo systemctl disable cron sudo systemctl enable cron</pre> Change crontab to ***** /usr/local/bin/monitoring.sh to make it run every minute	